CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

CSUE301 – : Big Data Analytics

Practical File – ODD Semester 2026-27

StudentID: 24DIT021
Student Name: HIT GOAYNI

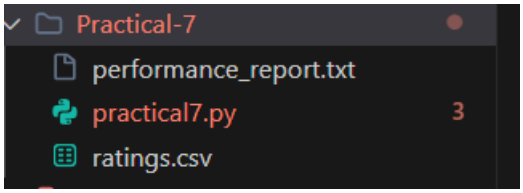
Practical 7

1. Problem Definition

You are a Big Data Engineer at MovieFlix Technologies, a movie recommendation company serving millions of users worldwide. The recommendation engine repeatedly processes movie ratings data to generate personalized suggestions. Due to the increasing volume of user interactions, query execution time has become a major concern. Management wants to optimize the Spark application using Caching and Partitioning techniques to reduce execution time and improve scalability.

PART A — Practical Execution

Step 1: Creating Folder and files



Step 2: Creating Spark Session

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Spark Performance Optimization") \
    .master("local[*]") \
    .getOrCreate()

print("Spark Session Created Successfully")
print("Spark Version:", spark.version)
```

```
============================================================
 >>> TASK 1: CREATING SPARK SESSION <<<
============================================================
WARNING: Using incubator modules: jdk.incubator.vector
26/09/27 16:07:26 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see
 https://cwiki.apache.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/27 16:07:26 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[*] Spark Version: 4.2.0
[*] Spark App Name: MovieFlix-Performance-Optimization
[*] Master URL: local[*]
```

```
df = spark.read.csv(
    "ratings.csv",
    header=True,
    inferSchema=True
)

print("Ratings Dataset Loaded Successfully")
```

```
================================================================================
 >>> TASK 2: LOADING DATASET INTO DATAFRAME <<<
================================================================================
[*] Loading CSV data from: C:\Users\ADMIN\OneDrive\Desktop\BDA\Practical-7\ratings.csv
[*] Dataset successfully loaded.

Sample Records (First 5 rows):
+------+-------+------+
|userId|movieId|rating|
+------+-------+------+
|   913|     13|   3.0|
|  2007|    115|   2.0|
|   840|    347|   5.0|
|   713|    303|   4.0|
|   261|     16|   1.5|
+------+-------+------+
only showing top 5 rows
```

Step 3: Loading Ratings Dataset

Step 4: Display Dataset Schema

```
================================================================================
 >>> TASK 3: DATASET SCHEMA & DATA TYPES <<<
================================================================================
DataFrame Schema:
root
 |-- userId: integer (nullable = true)
 |-- movieId: integer (nullable = true)
 |-- rating: double (nullable = true)
```

```
================================================================================
 >>> TASK 4: RECORD COUNT & INITIAL PARTITIONS <<<
================================================================================
[*] Total Number of Records: 300,000
[*] Initial Partition Count : 1
```

Step 5: Display Record Count and Initial Partitions

```
======================================================================
 >>> TASK 5 & 6: AVERAGE RATINGS & EXECUTION TIME BEFORE CACHING <<<
======================================================================
[*] Overall Average Rating across all movies: 3.00 / 5.00

Computing movie-wise average ratings (Uncached)...

Top 5 Most Rated Movies (Before Caching):
Movie ID   | Avg Rating   | Total Ratings
-----------------------------------------
393        | 2.98         | 680
187        | 2.96         | 674
210        | 2.98         | 663
327        | 3.06         | 661
75         | 3.01         | 661

[>>>] Execution Time BEFORE Caching: 0.8776 seconds
```

Step 6: Calculate Average Movie Rating

```
[>>>] Execution Time BEFORE Caching: 0.8776 seconds
```

Step 7: Measure Execution Time Before Caching

```
======================================================================
 >>> TASK 7 & 8: APPLYING CACHE() AND TRIGGERING CACHING <<<
======================================================================
[*] Applied df.cache() (StorageLevel: MEMORY_AND_DISK_DESER)
[*] Count used to trigger caching: 300,000 records
[*] Time taken to populate cache: 0.7008 seconds
[*] DataFrame is now cached in Spark Memory.
```

Step 8: Apply cache() and Trigger Caching

Step 9: Recalculate Average Ratings After Caching

```
======================================================================
 >>> TASK 9 & 10: RECALCULATING AVERAGE RATINGS AFTER CACHING <<<
======================================================================

Top 5 Most Rated Movies (After Caching):
Movie ID   | Avg Rating   | Total Ratings
----------------------------------------
393        | 2.98         | 680
187        | 2.96         | 674
210        | 2.98         | 663
327        | 3.06         | 661
75         | 3.01         | 661

[>>>] Execution Time AFTER Caching : 0.2630 seconds
```

```
================================================================
 >>> TASK 13: CACHING PERFORMANCE COMPARISON <<<
================================================================
Execution Time Before Cache : 0.8776 s
Execution Time After Cache  : 0.2630 s
Performance Speedup Factor  : 3.34x faster
Time Reduction Percentage   : 70.04%
```

Step 10: Measure Execution Time After Caching

Step 11: Repartition Dataset into 2, 4 and 8 Partitions

The dataset was repartitioned into 2, 4 and 8 partitions to compare execution performance.

Results:

Number of Partitions	Actual Partition Count	Execution Time
2	2	0.7512 s
4	4	0.4433 s
8	8	0.4900 s

The 4-partition configuration recorded the lowest execution time among these three measured repartitioning runs.

```
================================================================
 >>> TASK 11 & 12: REPARTITIONING (2, 4, AND 8 PARTITIONS) <<<
================================================================
[*] Repartitioned to 2 Partitions  | Actual Count: 2 | Execution Time: 0.7512 s
[*] Repartitioned to 4 Partitions  | Actual Count: 4 | Execution Time: 0.4433 s
[*] Repartitioned to 8 Partitions  | Actual Count: 8 | Execution Time: 0.4900 s
```

Step 12: Compare coalesce() and repartition()

```
================================================================
 >>> SUPPLEMENTARY 1: COALESCE() VS REPARTITION() <<<
================================================================
coalesce() avoids full network shuffle by merging existing partitions on same nodes.
Reducing partitions from 8 down to 2 using coalesce()...
[*] coalesce() Partition Count     : 2
[*] coalesce() Execution Time      : 0.4445 s
[*] repartition(2) Execution Time  : 0.7512 s
```

```
======================================================================
 >>> SUPPLEMENTARY 2: PERSIST(STORAGELEVEL.DISK_ONLY) <<<
======================================================================
Persisting DataFrame using StorageLevel.DISK_ONLY...
[*] Execution Time with DISK_ONLY  : 0.1475 s
[*] Execution Time with cache()    : 0.2630 s
```

Step 13: Compare cache() and persist(DISK_ONLY)

Step 14: Measure Multiple Spark Actions

Spark Action	Result	Execution Time
count()	300,000 rows	0.1262 s
take(10)	10 rows	0.0426 s
filter(rating >= 4.0)	99,945 rows	0.2983 s

```
======================================================================
 >>> SUPPLEMENTARY 3: MULTIPLE SPARK ACTIONS TIMING <<<
======================================================================
[*] Action count()               : 300,000 rows returned in 0.1262 s
[*] Action take(10)              : 10 rows returned in 0.0426 s
[*] Action filter(rating>=4.0)   : 99,945 rows matched in 0.2983 s
```

Step 15: Final Performance Summary

Dataset Details

File Name : ratings.csv

Total Records : 300,000

Schema : userId (int), movieId (int), rating (double)

Initial Partitions : 1

Overall Average Rating : 3.00 / 5.00

Caching Benchmark

Before Caching Time : 0.8776 seconds

After Caching Time : 0.2630 seconds

Performance Speedup : 3.34x faster

Time Saved : 70.04%

Partitioning Benchmark

2 Partitions : 0.7512 seconds

4 Partitions : 0.4433 seconds

8 Partitions : 0.4900 seconds **Repartition vs Coalesce** repartition(2) : 0.7512 seconds coalesce(2) : 0.4445 seconds **Storage Level Comparison** cache() : 0.2630 seconds persist(DISK_ONLY) : 0.1475 seconds

Comparison

```
================================================================================
                MOVIEFLIX SPARK PERFORMANCE OPTIMIZATION REPORT
================================================================================
1. DATASET DETAILS:
   - File Name                : ratings.csv
   - Total Records            : 300,000
   - Schema                   : userId (int), movieId (int), rating (double)
   - Initial Partitions       : 1
   - Overall Average Rating   : 3.00 / 5.00

2. CACHING BENCHMARK:
   - Before Caching Time      : 0.8776 seconds
   - Time Saved               : 70.04%

3. PARTITIONING BENCHMARK:
   - 2 Partitions Execution   : 0.7512 seconds
   - 4 Partitions Execution   : 0.4433 seconds
   - 8 Partitions Execution   : 0.4900 seconds

4. REPARTITION vs COALESCE:
   - repartition(2) Time      : 0.7512 seconds (Full Shuffle)
   - coalesce(2) Time         : 0.4445 seconds (No Full Shuffle)

5. STORAGE LEVEL COMPARISON:
   - cache() (Memory & Disk)  : 0.2630 seconds
   - persist(DISK_ONLY)       : 0.1475 seconds

6. OBSERVATIONS & KEY FINDINGS:
   a) Caching drastically cuts repetitive calculation latency by keeping pre-parsed
      records in executor memory, eliminating repetitive disk read and parsing overhead.
   b) Partitioning allows Spark to execute tasks concurrently across CPU cores. Having
      partitions aligned with available CPU cores achieves optimal task balancing.
   c) coalesce() is significantly more lightweight than repartition() when downsizing
      partitions because it avoids expensive cross-node shuffles.
   d) In-memory caching outperforms DISK_ONLY persist because RAM I/O has orders of
      magnitude higher throughput and lower latency than disk read/write cycles.

7. CONCLUSION:
   For MovieFlix recommendation pipelines, combining DataFrame cache() for iterative
   queries with appropriate partition sizing achieves maximum throughput and minimal
   query latency.
================================================================================
```

Practical-7 > performance_report.txt

```
================================================================================
            MOVIEFLIX SPARK PERFORMANCE OPTIMIZATION REPORT
================================================================================
1. DATASET DETAILS:
   - File Name                : ratings.csv
   - Total Records            : 300,000
   - Schema                   : userId (int), movieId (int), rating (double)
   - Initial Partitions       : 1
   - Overall Average Rating   : 3.00 / 5.00

2. CACHING BENCHMARK:
   - Before Caching Time      : 0.8776 seconds
   - After Caching Time       : 0.2630 seconds
   - Performance Speedup      : 3.34x faster
   - Time Saved               : 70.04%

3. PARTITIONING BENCHMARK:
   - 2 Partitions Execution   : 0.7512 seconds
   - 4 Partitions Execution   : 0.4433 seconds
   - 8 Partitions Execution   : 0.4900 seconds

4. REPARTITION vs COALESCE:
   - repartition(2) Time      : 0.7512 seconds (Full Shuffle)
   - coalesce(2) Time         : 0.4445 seconds (No Full Shuffle)

5. STORAGE LEVEL COMPARISON:
   - cache() (Memory & Disk)  : 0.2630 seconds
   - persist(DISK_ONLY)       : 0.1475 seconds

6. OBSERVATIONS & KEY FINDINGS:
   a) Caching drastically cuts repetitive calculation latency by keeping pre-parsed
      records in executor memory, eliminating repetitive disk read and parsing overhead.
   b) Partitioning allows Spark to execute tasks concurrently across CPU cores. Having
      partitions aligned with available CPU cores achieves optimal task balancing.
   c) coalesce() is significantly more lightweight than repartition() when downsizing
      partitions because it avoids expensive cross-node shuffles.
   d) In-memory caching outperforms DISK_ONLY persist because RAM I/O has orders of
      magnitude higher throughput and lower latency than disk read/write cycles.

```

Step 16: Performance Report

3. Key Questions / Analysis

Q1. What is caching in Apache Spark and why does it improve performance?

Caching is an optimization technique in Apache Spark in which a DataFrame or RDD is stored for reuse after it has been computed. When the same data is required again, Spark can reuse the cached data instead of recomputing it. This can reduce execution time for repeated operations.

Q2. What is the difference between cache() and persist()?

cache() is a convenient method for caching a DataFrame or RDD using Spark's default caching storage level.

persist() provides more control because a specific storage level can be selected. For example:

df.cache()

uses the default caching storage level, whereas:

df.persist(StorageLevel.DISK_ONLY)

stores the persisted data on disk.

Therefore:

cache() → Simple/default caching

persist() → Allows selection of storage level

Q3. How does Spark partition data internally?

Spark divides a dataset into smaller units called **partitions**. Spark can process these partitions through separate tasks, allowing data processing to be performed in parallel.

In this practical, the initial dataset contained **1 partition** and was later repartitioned into **2, 4 and 8 partitions**.

Q4. Why does repartition() improve scalability?

repartition() changes the number of partitions by redistributing data. It can increase parallelism when the existing number of partitions is not suitable for the workload.

For example:

df.repartition(8)

creates 8 partitions.

However, repartitioning involves data redistribution and therefore can introduce shufle overhead.

Q5. How does lazy evaluation affect Spark performance?

Spark uses lazy evaluation for transformations. Operations such as select(), filter() and withColumn() do not immediately execute the computation.

Spark builds an execution plan and executes the required operations when an action such as count(), show() or collect() is called.

Therefore:

Transformation → Defines the

operation Action → Triggers execution

4. Conclusion

The practical successfully demonstrated Spark performance optimization using **Caching and Partitioning** techniques. The movie ratings dataset containing **300,000 records** was loaded into a Spark DataFrame, its schema and partition count were examined, and the average movie rating was calculated.

The execution time before caching was **0.8776 seconds**, while the measured execution time after caching was **0.2630 seconds**. This resulted in a measured **3.34× speedup** and **70.04% time reduction** for the tested operation.

The dataset was also repartitioned into **2, 4 and 8 partitions** to study the effect of partitioning on execution time. The practical further compared coalesce() with repartition() and cache() with persist(StorageLevel.DISK_ONLY).

The experiment demonstrated how caching can reduce repeated computation and how partitioning techniques can be used to control parallel data processing in Spark.